

BASIC NETWORK SNIFFER USING PYTHON AND SCAPY

INTRODUCTION

Network traffic analysis is an essential aspect of cybersecurity and network administration. Understanding how data travels across a network helps security professionals identify malicious activities, troubleshoot connectivity issues, and gain insight into network behavior. Packet sniffing is the process of capturing and inspecting data packets as they travel through a network. This project focuses on developing a basic network sniffer using Python and the Scapy library. The program captures live network packets and extracts useful information such as source and destination IP addresses, protocols, and packet payloads. Through this project, the fundamentals of network communication and protocol analysis are explored.

AIM AND OBJECTIVES

Aim

To develop a Python-based network packet sniffer capable of capturing and analyzing network traffic in real time.

Objectives

- To capture live network packets using Python
 - To analyze packet structures and contents
 - To understand how data flows through a network
 - To identify common network protocols
 - To display useful packet information such as IP addresses, protocols, and payloads
-

TOOLS AND ENVIRONMENT

The following tools and technologies were used:

- Python 3
- Scapy Library
- Kali Linux
- Terminal Environment

Scapy was selected because it provides powerful packet manipulation and packet capture capabilities suitable for network analysis tasks.

```
(sambi@kali)-[~]
└─$ sudo apt update
[sudo] password for sambi:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.2 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [104 kB]
Fetched 74.7 MB in 55s (1,355 kB/s)
2157 packages can be upgraded. Run 'apt list --upgradable' to see them.

(sambi@kali)-[~]
└─$ python3
Python 3.12.7 (main, Nov 8 2024, 17:55:36) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
KeyboardInterrupt
>>> exit
Use exit() or Ctrl-D (i.e. EOF) to exit
>>>

(sambi@kali)-[~]
└─$ nano packet_sniffer.py

(sambi@kali)-[~]
└─$ ls
attack_capture.pcap  Desktop  evilginx2  head  loop.sh  my_likes.txt  pPPs.txt  report.txt  testfile.pdf  ZPhisher
beef                 'digital forensics'  fsociety  herny-01.cap  MedusaPhisher  packet_sniffer.py  private_key.pem  signature  testfile.txt
blue.nmap            Downloads  gr33n37-ip-changer  IMEI-TRACKER  mand-01.cap  passwords.txt  prrrratic.pcapng  t  TT.txt
clean.b64            Downloads  hash-id.py  Install-DVWA.sh  mindou-01.cap  pentestlab  public_key.pem  target.txt  Ubun-01.cap
cron-lab             evebox    hashme.txt  locker_script.sh  Music  Pictures  public_key.pem  Templates  Videos

(sambi@kali)-[~]
└─$ rm packet_sniffer.py

(sambi@kali)-[~]
└─$ ls
```

✦ Figure 1: *Successful installation of Scapy or python3 in terminal*

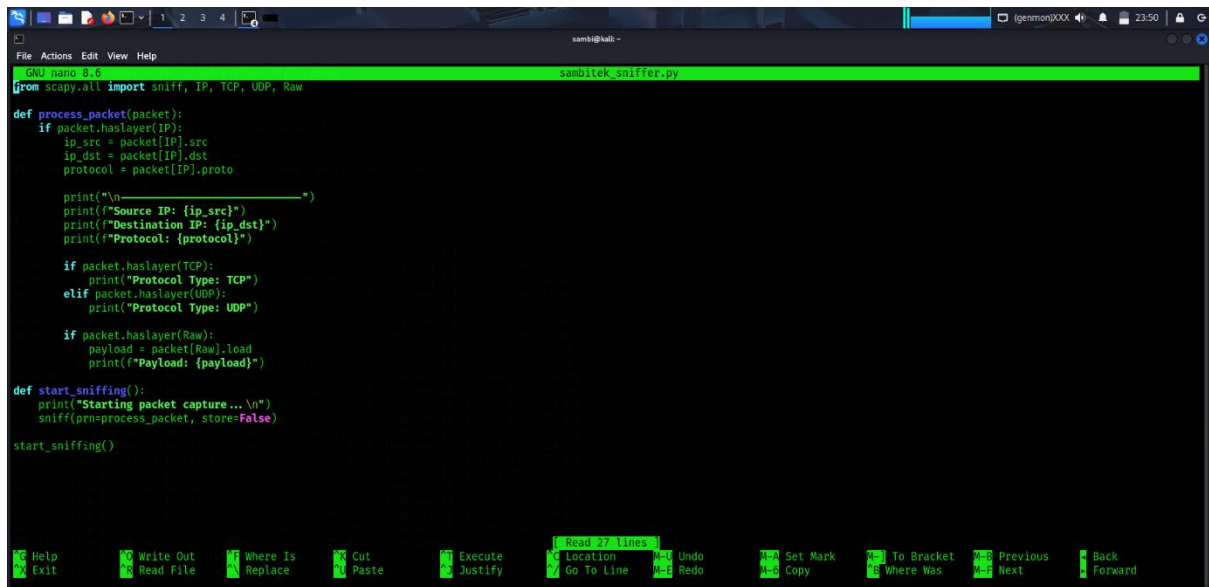
PROGRAM DEVELOPMENT

A Python script was developed using the Scapy library to capture network packets and extract relevant information.

The program performs the following functions:

- Captures packets from the network interface
- Identifies IP packets
- Displays source and destination IP addresses
- Detects TCP and UDP protocols
- Extracts and displays packet payloads when available

The core packet capture function continuously listens for network traffic and processes packets in real time.



```
File Actions Edit View Help
GNU nano 2.9.3 sambitek_sniffer.py
from scapy.all import sniff, IP, TCP, UDP, Raw

def process_packet(packet):
    if packet.haslayer(IP):
        ip_src = packet[IP].src
        ip_dst = packet[IP].dst
        protocol = packet[IP].proto

        print("\n-----")
        print(f"Source IP: {ip_src}")
        print(f"Destination IP: {ip_dst}")
        print(f"Protocol: {protocol}")

        if packet.haslayer(TCP):
            print("Protocol Type: TCP")
        elif packet.haslayer(UDP):
            print("Protocol Type: UDP")

        if packet.haslayer(Raw):
            payload = packet[Raw].load
            print(f"Payload: {payload}")

def start_sniffing():
    print("Starting packet capture...\n")
    sniff(prn=process_packet, store=False)

start_sniffing()

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Read 27 lines Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was Previous Next Back Forward
```

✦ Figure 2: Complete Python packet sniffer code

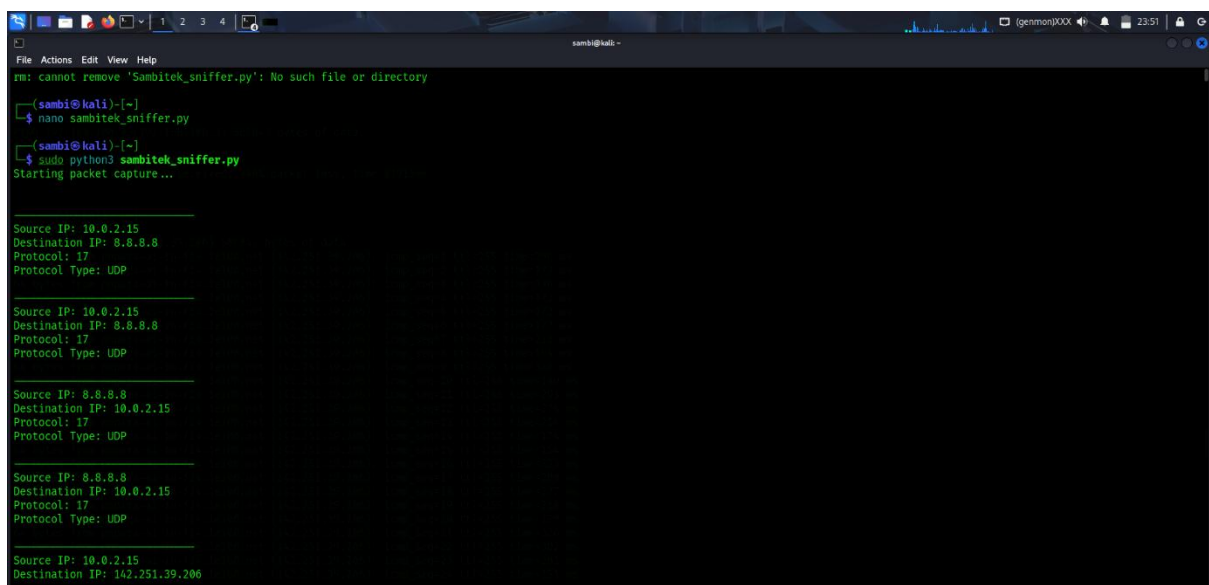
PACKET CAPTURE EXECUTION

The packet sniffer was executed with administrative privileges to allow access to network interfaces.

Command used:

```
sudo python3 sambitek_sniffer.py
```

Once launched, the program continuously monitored network traffic and displayed captured packet information on the terminal.



```
File Actions Edit View Help
rm: cannot remove 'Sambitek_sniffer.py': No such file or directory
(sambi@kali)~$ nano sambitek_sniffer.py
(sambi@kali)~$ sudo python3 sambitek_sniffer.py
Starting packet capture...

Source IP: 10.0.2.15
Destination IP: 8.8.8.8
Protocol: 17
Protocol Type: UDP

Source IP: 10.0.2.15
Destination IP: 8.8.8.8
Protocol: 17
Protocol Type: UDP

Source IP: 8.8.8.8
Destination IP: 10.0.2.15
Protocol: 17
Protocol Type: UDP

Source IP: 8.8.8.8
Destination IP: 10.0.2.15
Protocol: 17
Protocol Type: UDP

Source IP: 10.0.2.15
Destination IP: 142.251.39.206
```

✦ *Figure 3: Packet sniffer running successfully in terminal*

TRAFFIC GENERATION

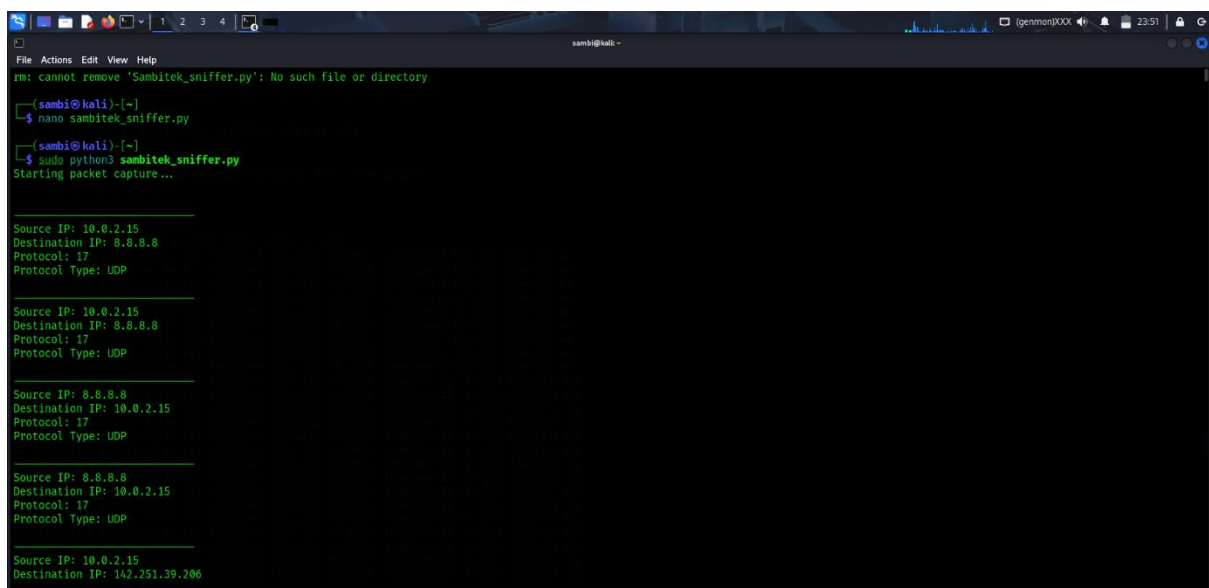
To generate network traffic for analysis, several activities were performed, including:

- Visiting websites through a web browser
- Sending ICMP requests using the ping command
- Performing network scans using Nmap

These activities generated packets that were captured and analyzed by the packet sniffer.

Example command used:

ping google.com



```
File Actions Edit View Help
rm: cannot remove 'Sambitek_sniffer.py': No such file or directory

(sambi@kali)-[~]
└─$ nano sambitek_sniffer.py

(sambi@kali)-[~]
└─$ sudo python3 sambitek_sniffer.py
Starting packet capture...

Source IP: 10.0.2.15
Destination IP: 8.8.8.8
Protocol: 17
Protocol Type: UDP

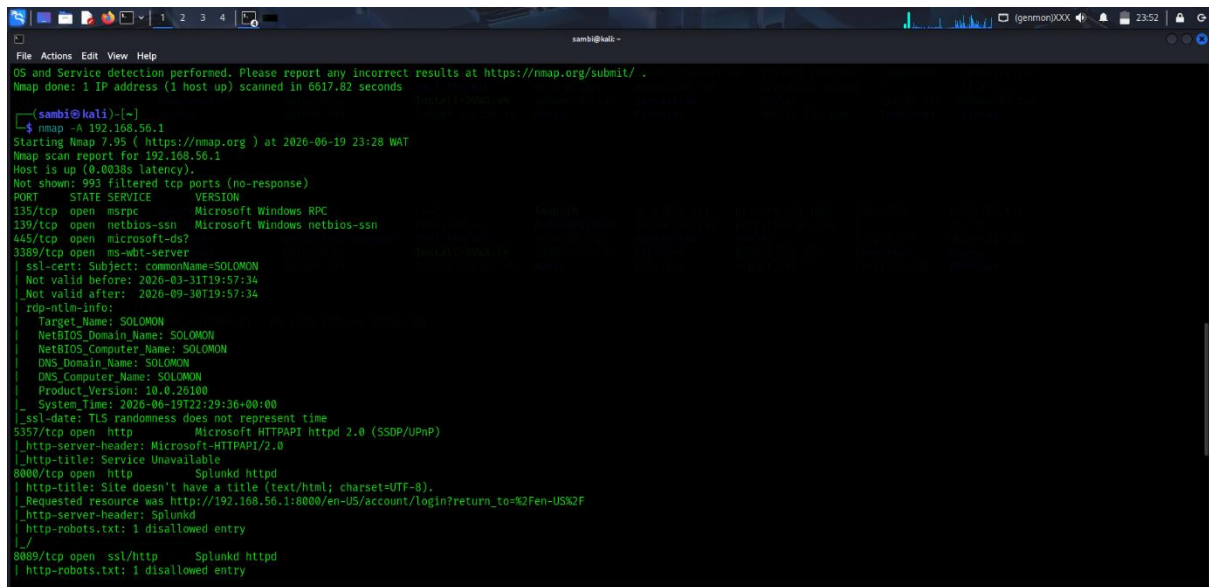
Source IP: 10.0.2.15
Destination IP: 8.8.8.8
Protocol: 17
Protocol Type: UDP

Source IP: 8.8.8.8
Destination IP: 10.0.2.15
Protocol: 17
Protocol Type: UDP

Source IP: 8.8.8.8
Destination IP: 10.0.2.15
Protocol: 17
Protocol Type: UDP

Source IP: 10.0.2.15
Destination IP: 142.251.39.206
```

✦ *Figure 4 Traffic generation using ping command*



```
(sambi@kali)-[~]
└─$ nmap -A 192.168.56.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-19 23:28 WAT
Nmap scan report for 192.168.56.1
Host is up (0.0030s latency).
Not shown: 993 filtered tcp ports (no-response)
PORT      STATE SERVICE        VERSION
135/tcp   open  msrpc          Microsoft Windows RPC
139/tcp   open  netbios-ssn    Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds?
3389/tcp  open  ms-wbt-server
|_ ssl-cert: Subject: commonName=SOLOMON
|_ Not valid before: 2026-03-31T19:57:34
|_ Not valid after: 2026-09-30T19:57:34
|_ rdp-ntlm-info:
|   Target_Name: SOLOMON
|   NetBIOS_Domain_Name: SOLOMON
|   NetBIOS_Computer_Name: SOLOMON
|   DNS_Domain_Name: SOLOMON
|   DNS_Computer_Name: SOLOMON
|   Product_Version: 10.0.26100
|_ System_Time: 2026-06-19T22:29:36+00:00
|_ ssl-date: TLS randomness does not represent time
5357/tcp  open  http           Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ http-server-header: Microsoft-HTTPAPI/2.0
|_ http-title: Service Unavailable
8000/tcp  open  http           Splunkd httpd
|_ http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_ Requested resource was http://192.168.56.1:8000/en-US/account/login?return_to=%2Fen-US%2F
|_ http-server-header: Splunkd
|_ http-robots.txt: 1 disallowed entry
|_
8089/tcp  open  ssl/http       Splunkd httpd
|_ http-robots.txt: 1 disallowed entry
```

✦ *Figure 5: Nmap scan or web browsing activity used to generate traffic*

PACKET ANALYSIS

The captured packets revealed important network information, including:

Source and Destination IP Addresses

The program successfully identified the originating and receiving IP addresses of captured packets. This information helps track communication between devices on a network.

Protocol Identification

The packet sniffer detected common protocols such as:

- TCP (Transmission Control Protocol)
- UDP (User Datagram Protocol)

Protocol identification provides insight into the type of communication occurring on the network.

Payload Analysis

For packets containing data, the payload was displayed. This demonstrated how application data is transmitted across networks and how packet contents can be inspected.

```

Source IP: 10.0.2.15
Destination IP: 192.168.56.1
Protocol: 6
Protocol Type: TCP

Source IP: 10.0.2.15
Destination IP: 192.168.56.1
Protocol: 6
Protocol Type: TCP

Source IP: 142.251.39.206
Destination IP: 10.0.2.15
Protocol: 1
Payload: b'\xaf\xc25j\x00\x00?\xb0\t\x00\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&'()*+,-./01234567'

Source IP: 192.168.56.1
Destination IP: 10.0.2.15
Protocol: 6
Protocol Type: TCP

Source IP: 192.168.56.1
Destination IP: 10.0.2.15
Protocol: 6
Protocol Type: TCP

Source IP: 192.168.56.1
Destination IP: 10.0.2.15
Protocol: 6
Protocol Type: TCP

```

✦ *Figure 6: Terminal output showing source IP, destination IP, and protocol information*

```

Protocol: 1
Payload: b'\xf8\xc25j\x00\x00\x00\x00\x04\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&'()*+,-./01234567'

Source IP: 142.251.39.206
Destination IP: 10.0.2.15
Protocol: 1
Payload: b'\xf8\xc25j\x00\x00\x00\x00\x04\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&'()*+,-./01234567'

Source IP: 10.0.2.15
Destination IP: 142.251.39.206
Protocol: 1
Payload: b'\xf9\xc25j\x00\x00\x00\x00\x04\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&'()*+,-./01234567'

Source IP: 142.251.39.206
Destination IP: 10.0.2.15
Protocol: 1
Payload: b'\xf9\xc25j\x00\x00\x00\x00\x04\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&'()*+,-./01234567'

Source IP: 10.0.2.15
Destination IP: 142.251.39.206
Protocol: 1
Payload: b'\xfa\xc25j\x00\x00\x00\x00\x00wz\x04\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&'()*+,-./01234567'

Source IP: 142.251.39.206
Destination IP: 10.0.2.15
Protocol: 1
Payload: b'\xf8\xc25j\x00\x00\x00wz\x04\x00\x00\x00\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f !"#%&'()*+,-./01234567'

Source IP: 10.0.2.15
Destination IP: 142.251.39.206
Protocol: 1

```

✦ *Figure 7: Packet output displaying payload data*

UNDERSTANDING NETWORK DATA FLOW

The project provided practical insight into how information flows across a network. When a user accesses a website or sends a request, data is divided into packets and transmitted between devices. Each packet contains addressing information and protocol details that enable successful communication.

By observing captured packets, it was possible to understand:

- How devices communicate using IP addresses
- How protocols facilitate data transmission

- How requests and responses travel between systems
 - How packet payloads carry application data
-

FINDINGS

The following observations were made during the project:

- Network traffic can be captured and analyzed using Python and Scapy
 - Source and destination IP addresses provide visibility into network communications
 - TCP and UDP are among the most commonly observed protocols
 - Packet payloads reveal the actual data being transmitted
 - Packet sniffing provides valuable insight into network behavior and security monitoring
-

CONCLUSION

This project successfully demonstrated the development of a basic network sniffer using Python and Scapy. The application captured live network traffic and displayed essential packet information, including IP addresses, protocols, and payloads. The exercise provided practical experience in packet analysis and improved understanding of network communication processes. It also highlighted the importance of packet monitoring in cybersecurity, network troubleshooting, and traffic analysis. The project serves as a foundation for more advanced network monitoring and intrusion detection techniques.